

Arquitectura del Sistema — Inmotika

Visión general

Inmotika es una plataforma SaaS de gestión inmobiliaria y domótica integrada. Digitaliza la gestión de propiedades, visitas técnicas, dispositivos, protocolos e informes en un sistema multirol seguro.

Stack tecnológico

Capa	Tecnología	Versión
Frontend	React	19
Build tool	Vite	7
Estilos	Tailwind CSS	4
Iconos	Lucide React	—
Backend / DB	Supabase (PostgreSQL)	—
Auth	Supabase Auth	—
Storage	Supabase Storage	—
Edge Functions	Deno (Supabase Functions)	—
Email	Resend (vía SMTP y API)	—
PDF	@react-pdf/renderer	4
Analytics	Vercel Analytics	—
Deploy	Vercel	—

Estructura de carpetas — Frontend

```
frontend/src/
├─ api/                                # Acceso a datos por entidad (Supabase queries)
│   ├── clienteApi.js
│   ├── visitaApi.js
│   ├── informeApi.js
│   └─ ...
├─ components/
│   ├── ui/                            # Primitivos reutilizables (Button, Card, Input, Modal...)
│   ├── forms/                         # Formularios compuestos (BranchForm, LocationPickerRows...)
│   ├── configuration/                 # Navegadores y modales de configuración
│   ├── shared/                       # Componentes compartidos entre módulos
│   └─ visits/                        # Componentes específicos de visitas e informes
├─ context/                           # React Context providers
│   ├── AuthContext.jsx               # Usuario autenticado y rol
│   └─ MasterDataContext.jsx         # Datos maestros (clientes, sucursales, dispositivos)
```

```

|   └─ NotificationContext.jsx
|   └─ ConfirmContext.jsx
└─ hooks/                # Lógica de negocio y fetching
    └─ useVisitas.js      # Gestión de visitas (coordinador/director/técnico)
    └─ useVisitasCliente.js # Vista de visitas para el cliente
    └─ useEmail.js        # Helpers de email (ya no resuelve destinatarios)
    └─ useSolicitudesVisita.js
    └─ ...
└─ modules/              # Vertical slices por dominio
    └─ clients/          # ClientForm, ClientsView
    └─ contacts/         # ContactForm, ContactNavigator
    └─ devices/          # DeviceForm, CategoriaForm
    └─ visits/           # ProgramacionForm, SolicitudVisitaPage
└─ pages/                # Puntos de entrada por rol
    └─ LoginPage.jsx
    └─ DashboardPage.jsx
    └─ ConfigurationPage.jsx
    └─ SchedulePage.jsx
    └─ ClientDataPage.jsx
    └─ ClientInventoryPage.jsx
    └─ ClientVisitsPage.jsx
    └─ ClientDashboardPage.jsx
    └─ MensajesPage.jsx
    └─ CategoriasPage.jsx
    └─ visits/           # GestionVisitasPage, SolicitudVisitaPage
    └─ informe/          # InformeRevisionPage, InformeDirectorPage, ValidacionInformePage
└─ utils/
    └─ supabase.js       # Cliente Supabase (anon key)
    └─ constants.js      # ROLES, ROLE_HIERARCHY
    └─ validators.js     # Validaciones de formularios
    └─ entityMappers.js  # Mapeo BD ↔ drafts de formulario
    └─ generateInforme.js # Generación y subida de PDF
    └─ locationData.js   # Datos estáticos de países/estados/ciudades (reemplaza country-
state-city)
└─ assets/              # Imágenes y fuentes

```

Estructura de carpetas — Backend

```

supabase/
└─ functions/
    └─ send-email/       # Envío de correos transaccionales (Resend)
        └─ index.ts      # Handler + resolución de destinatarios con service_role
        └─ templates.ts  # Plantillas HTML de todos los tipos de correo
    └─ invite-user/      # Invitación de usuarios al sistema
    └─ download-informe/ # Descarga segura de PDFs de informes
    └─ check-director-timeout/ # Cron: verifica timeout de aprobación director
Database/
└─ ddl.sql              # Fuente de verdad del esquema completo
└─ DOCUMENTACION_BD.md # Documentación ampliada de la base de datos

```

Routing

No usa React Router. El routing es **tab-based** controlado por estado en `App.jsx` . Cada rol ve un conjunto diferente de tabs/páginas.

Autenticación

- Supabase Auth con email/password
 - Invitación por email (Edge Function `invite-user`)
 - Al crear usuario en `auth.users` , el trigger `handle_new_user` crea automáticamente:
 - `perfil_usuario` con el rol asignado
 - Fila en la tabla especializada del rol (`tecnico` , `coordinador` , `director`)
 - El rol se cambia actualizando `perfil_usuario.rol_id` — el trigger `sync_specialized_role_tables` sincroniza las tablas especializadas
-

Storage — Buckets

Bucket	Tipo	Contenido
inmotika	Privado	Informes PDF, firmas, avatares, documentos usuarios
assets	Público	Logo y plantilla de hoja (<code>logo_inmotika.png</code> , <code>hoja_fondo.png</code>)

Rutas dentro del bucket `inmotika`

```
informes/{visita_id}/informe.pdf
evidencias/{visita_id}/{dispositivo_id}/etiqueta.{ext}
evidencias/{visita_id}/{dispositivo_id}/foto_{n}.{ext}
usuarios/{usuario_id}/avatar
usuarios/{usuario_id}/firma.{ext}
usuarios/{usuario_id}/docs/{nombre}.{ext}
```

Variables de entorno

Frontend (`.env`)

```
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
VITE_LOGO_URL=https://...supabase.co/storage/v1/object/public/assets/logo_inmotika.png
VITE_FONDO_URL=https://...supabase.co/storage/v1/object/public/assets/hoja_fondo.png
VITE_APP_URL=https://inmotika.vercel.app
```

Edge Functions (Supabase Secrets)

```
RESEND_API_KEY=
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
```

Patrones clave

Emails — arquitectura servidor

Todos los emails se envían a través de la Edge Function `send-email`. El frontend **nunca resuelve destinatarios** — solo pasa IDs de contexto (`visitaId`, `sucursalId`, `clienteId`). La Edge Function usa `service_role` para resolver destinatarios sin restricciones RLS.

Generación de PDF

`@react-pdf/renderer` corre en un Web Worker. Las URLs de imágenes (logo, fondo, firmas) deben pasarse como props explícitos — `import.meta.env` no está disponible en el worker.

Datos de ubicación

La librería `country-state-city` (8.7 MB) fue reemplazada por `src/utils/locationData.js` — datos estáticos con todos los países del mundo y estados/ciudades solo de Latinoamérica (~360 KB).

Triggers — no duplicar lógica

- `handle_new_user` : NO insertar manualmente en `perfil_usuario` ni tablas de rol al registrar usuarios
- `sync_specialized_role_tables` : para cambiar rol, solo actualizar `perfil_usuario.rol_id`
- `handle_updated_at` : actualiza `updated_at` automáticamente en todas las tablas que lo tienen